UNIVERSIDADE
FEDERAL DO CEARÁ

Relatório Final

Disciplina: Qualidade de Software
Equipe: Levi Gomes Ferreira (matrícula 556935) e Pablo Vinicios da Silva Araújo (matrícula 574229)
Projeto: Camelot — https://github.com/camelot-dev/camelot
***Fork*:** https://github.com/PabloVini28/camelot
***Pull Request*:** https://github.com/camelot-dev/camelot/pull/871
Commit SHA: b300e9e31a01ec3bb00580245a208634b9f9aefb

1. Apresentação do projeto

O Camelot (https://github.com/camelot-dev/camelot) é uma biblioteca Python para extrair tabelas de arquivos PDF. Ela tem cinco parsers: *lattice* (tabelas com linhas de grade), *stream* (tabelas separadas por espaços), *network* e *hybrid* (alinhamento do texto) e ml (modelo neural opcional), além de *flavor*="auto", que escolhe um deles. Cada tabela extraída vira um *DataFrame* do pandas e pode ser exportada para CSV, JSON, Excel, HTML, *Markdown* e *SQLite*. O público são pessoas que precisam tirar dados tabulares de relatórios em PDF: analistas de dados, jornalistas, pesquisadores e desenvolvedores que automatizam esse tipo de extração.

Os testes usam *pytest* e ficam na pasta tests/: 29 módulos test_*.py organizados por funcionalidade (parsers, interface de linha de comando, exportação, backends de conversão de imagem, utilitários, erros e avisos), o *conftest.py*, com fixtures compartilhadas como *testdir* e *foo_pdf* e marcadores como *skip_on_windows*, o *data.py*, com as tabelas esperadas usadas nas comparações, e a pasta tests/files, com os PDFs de entrada. No commit-base (b300e9e) a suíte tinha 318 testes aprovados, 7 ignorados, 2 falhas esperadas (xfail) e 1 xpass, com 88,51% de cobertura do pacote camelot/. O projeto exige cobertura mínima de 90% (*fail_under* = 90) e roda a suíte no CI com nox, em várias versões do Python e em Linux, Windows e macOS.

2. Estado inicial do projeto

2.1 Test smells detectados antes da refatoração (TEMPY)

Test smell	Ocorrências detectadas
Unknown Test	54
Conditional Test Logic	7
Exception Handling	7

Test smell	Ocorrências detectadas
Verbose Test	3
Programming Paradigms Blend	2
Redundant Print	1
Non-Functional Statement	1
Total	**75**

2.2 Resultados dos testes e cobertura antes da refatoração (Pytest)

Métrica	Valor
Testes aprovados (Passed)	318
Testes com falha (Failed)	0
Erros (Errors)	0
Ignorados (Skipped)	7
Cobertura total (%)	88,51

2.3 Métricas do código de teste antes da refatoração (Radon)

Métrica	Valor
SLOC total dos testes (linha TOTAL com origem = teste)	8.741
Complexidade ciclomática média das funções de teste	2,449
Índice de manutenibilidade médio dos arquivos de teste	68,48

3. Refatorações realizadas

3.1 Seleção das ocorrências

Refatoramos 40 ocorrências de 7 tipos de test smell, priorizando os de severidade alta no CATSPY. Dos 75 smells que o TEMPY encontrou no commit-base, quatro tipos têm severidade alta no catálogo: Unknown Test (54 ocorrências), Exception Handling (7), Conditional Test Logic (7) e Verbose Test (3). Eles concentram 36 das 40 refatorações: todas as ocorrências de Exception Handling e de Conditional Test Logic, 2 das 3 de Verbose Test e 20 das 54 de Unknown Test.

As outras 4 refatorações zeraram os tipos de severidade média e baixa, que tinham poucas ocorrências: Programming Paradigms Blend (média, 2), Redundant Print (baixa, 1) e Non-Functional Statement (baixa, 1). Assim, no estado final só restam ocorrências de Unknown Test (34) e uma de Verbose Test.

A divisão entre os dois integrantes também orientou a escolha: cada um ficou com 20 ocorrências de 5 tipos, e cada método refatorado ficou com uma só pessoa, para evitar conflito entre commits.

3.2 Lista das refatorações

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R01	*Redundant Print*	tests/test_errors.py::test_pdfium_backend_import_error	524d75c	sim	sim	não
R02	*Exception Handling*	tests/test_errors.py::test_pdfium_backend_import_error	ec69cbd	sim	sim	não
R03	*Exception Handling*	tests/test_errors.py::test_no_tables_found_logs_suppressed	c20dbbb	sim	sim	não
R04	*Exception Handling*	tests/test_errors.py::test_no_tables_found_warnings_suppressed	25e536d	sim	sim	não
R05	*Exception Handling*	tests/test_cli.py::test_cli_quiet	d7482ec	sim	sim	não
R06	*Conditional Test Logic*	tests/test_cli.py::test_help_output	22dd2a4	sim	sim	não
R07	*Conditional Test Logic*	tests/test_vector_engine_probe.py::test_lattice_engine_validation	2c89c3b	sim	sim	sim: Unknown Test em test_lattice_engine_validation (temporário, removido no R08)
R08	*Conditional Test Logic*	tests/test_vector_engine_probe.py::test_lattice_engine_validation	7afa9a5	sim	sim	não (e remove o Unknown Test do R07)
R09	*Verbose Test*	tests/test_cli.py::test_cli_stream	165a9ad	sim	sim	não
R10	*Verbose Test*	tests/test_cli.py::test_cli_output_format	7da8ff0	sim	sim	não
R11	*Unknown Test*	tests/test_errors.py::test_unknown_flavor	f0de476	sim	sim	não
R12	*Unknown Test*	tests/test_errors.py::test_input_kwargs	6a34896	sim	sim	não
R13	*Unknown Test*	tests/test_errors.py::test_no_tables_found_logs_suppressed	97082a7	sim	sim	não
R14	*Unknown Test*	tests/test_errors.py::test_no_tables_found_warnings_suppressed	cb0727e	sim	sim	não
R15	*Unknown Test*	tests/test_errors.py::test_no_password	7823dcd	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R16	*Unknown Test*	tests/test_errors.py::test_bad_password	a519c29	sim	sim	não
R17	*Unknown Test*	tests/test_errors.py::test_stream_equal_length	c0ba461	sim	sim	não
R18	*Unknown Test*	tests/test_errors.py::test_lattice_unknown_backend	aa1b20d	sim	sim	não
R19	*Unknown Test*	tests/test_errors.py::test_extraction_not_allowed	b1f5f8e	sim	sim	não
R20	*Unknown Test*	tests/test_cli.py::test_cli_quiet	df7a11d	sim	sim	não
R21	*Exception Handling*	tests/test_cli.py::test_import_error	7dcce8e	sim	sim	não
R22	*Exception Handling*	tests/test_export.py::test_export_sqlite	83cde00	sim	sim	não
R23	*Exception Handling*	tests/test_export.py::test_table_to_sqlite	12a7d69	sim	sim	não
R24	*Conditional Test Logic*	tests/test_common.py::test_overlapping_text_preserves_adjacent_cell	c665e60	sim	sim	não
R25	*Conditional Test Logic*	tests/test_export.py::test_export_sqlite	e6d3ec4	sim	sim	não
R26	*Conditional Test Logic*	tests/test_utils.py::test_bbox_from_str_rejects_zero_area	01929b4	sim	sim	não
R27	*Conditional Test Logic*	tests/test_ml_backend.py::test_ml_ocr_requires_engine_when_missing	f681793	sim	sim	não
R28	*Unknown Test*	tests/test_per_page.py::test_per_page_invalid_kwarg_rejected	aabc69e	sim	sim	não
R29	*Unknown Test*	tests/test_per_page.py::test_per_page_invalid_flavor_rejected	b1b6dc5	sim	sim	não
R30	*Unknown Test*	tests/test_per_page.py::test_per_page_auto_flavor_rejected	5edfc76	sim	sim	não
R31	*Unknown Test*	tests/test_per_page.py::test_per_page_non_dict_value_rejected	db9a211	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R32	*Unknown Test*	tests/test_utils.py::test_bbox_from_str_rejects_zero_area	ba2fb37	sim	sim	não
R33	*Unknown Test*	tests/test_utils.py::test_bbox_from_str_rejects_malformed	3c106ed	sim	sim	não
R34	*Unknown Test*	tests/test_utils.py::test_image_bbox_to_pdf_rejects_degenerate_input	aa7cf9b	sim	sim	não
R35	*Unknown Test*	tests/test_ml_backend.py::test_ml_without_deps_raises_helpful_error	8e97f32	sim	sim	não
R36	*Unknown Test*	tests/test_ml_backend.py::test_ml_read_pdf_surfaces_install_hint	013bebb	sim	sim	não
R37	*Unknown Test*	tests/test_ml_backend.py::test_ml_ocr_requires_engine_when_missing	35ae7ee	sim	sim	não
R38	*Programming Paradigms Blend*	tests/test_image_conversion_backend.py (nível do módulo)	62adc07	sim	sim	não
R39	*Programming Paradigms Blend*	tests/test_utils.py (nível do módulo)	a6d8e22	sim	sim	não
R40	*Non-Functional Statement*	tests/test_errors.py::test_lattice_no_convert_method	41bfcdb	sim	sim	não

3.3 Exemplos detalhados

Os prints são da página de cada arquivo no GitHub do fork, no commit anterior (antes) e no próprio commit (depois), com o trecho alterado destacado em amarelo pelo próprio GitHub. A complexidade e o SLOC de cada exemplo foram medidos com o Radon no commit anterior e no próprio commit, para isolar o efeito daquela refatoração.

R01 – *Redundant Print* (tests/test_errors.py::test_pdfium_backend_import_error)

Commit 524d75c · Levi Gomes Ferreira

Antes:

```
162
163 ••• def test_pdfium_backend_import_error(testdir):
164         filename = os.path.join(testdir, "table_region.pdf")
165         with mock.patch.dict(sys.modules, {"pypdfium2": None}):
166             message = "pypdfium2 is not available: "
167             try:
168                 tables = camelot.read_pdf(
169                     filename,
170                     flavor="lattice",
171                     backend="pdfium",
172                     use_fallback=False,
173                 )
174             except Exception as em:
175                 print(em)
176                 assert message in str(em)
177
```

Depois:

```
162
163 ••• def test_pdfium_backend_import_error(testdir):
164         filename = os.path.join(testdir, "table_region.pdf")
165         with mock.patch.dict(sys.modules, {"pypdfium2": None}):
166             message = "pypdfium2 is not available: "
167             try:
168                 tables = camelot.read_pdf(
169                     filename,
170                     flavor="lattice",
171                     backend="pdfium",
172                     use_fallback=False,
173                 )
174             except Exception as em:
175                 assert message in str(em)
176
```

Por que era um *test smell*: O print(em) ficava dentro de um except. Em um teste, um print não verifica nada e só polui a saída; a informação útil já está na asserção da linha seguinte. Além disso, esse except nunca executava (o pypdfium2 já está importado quando o teste roda), então o print nunca aparecia.

Estratégia do CATSPY aplicada: Remover a instrução de saída e deixar a verificação só na asserção, como no exemplo refatorado do catálogo. Foi aplicada sem adaptação. O *try/except* do mesmo teste foi tratado à parte, no R02 (*Exception Handling*).

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	3	3
SLOC do arquivo de teste (raw_por_arquivo_e_total)	156	155

R03 – *Exception Handling* (*tests/test_errors.py::test_no_tables_found_logs_suppressed*)

Commit c20dbbb · Levi Gomes Ferreira

Antes:

```
44
45  @skip_on_windows
46  def test_no_tables_found_logs_suppressed(testdir):
47      filename = os.path.join(testdir, "foo.pdf")
48      with warnings.catch_warnings():
49          # the test should fail if any warning is thrown
50          warnings.simplefilter("error")
51          try:
52              camelot.read_pdf(filename, suppress_stdout=True)
53          except Warning as e:
54              warning_text = str(e)
55              pytest.fail(f"Unexpected warning: {warning_text}")
56
```

Depois:

```
44
45  @skip_on_windows
46  def test_no_tables_found_logs_suppressed(testdir):
47      filename = os.path.join(testdir, "foo.pdf")
48      with warnings.catch_warnings():
49          # the test should fail if any warning is thrown
50          warnings.simplefilter("error")
51          camelot.read_pdf(filename, suppress_stdout=True)
52
```

Por que era um test smell: A chamada ficava em um *try/except Warning* só para chamar pytest.fail. O tratamento manual de exceção esconde a intenção do teste e repete o que *warnings.simplefilter("error")* já faz: transformar qualquer aviso em erro.

Estratégia do CATSPY aplicada: O CATSPY sugere trocar o *try/except* pelo recurso de exceção do *framework* (*assertRaises* no unittest; no *pytest*, *pytest.raises*). Aqui o teste não espera exceção nenhuma, então *pytest.raises* não se aplica. A adaptação foi remover o *try/except* e deixar o *simplefilter("error")* fazer o teste falhar se surgir qualquer aviso.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	2	1
SLOC do arquivo de teste (raw_por_arquivo_e_total)	154	150

R06 – Conditional Test Logic (tests/test_cli.py::test_help_output)

Commit 22dd2a4 · Levi Gomes Ferreira

Antes:

```
14
15  def test_help_output():
16      runner = CliRunner()
17      prog_name = runner.get_default_prog_name(cli)
18      result = runner.invoke(cli, ["--help"])
19      output = result.output
20
21      assert prog_name == "camelot"
22      assert result.output.startswith(
23          "Usage: {prog_name} [OPTIONS] COMMAND".format(**locals())
24      )
25      assert all(
26          v in result.output
27          for v in ["Options:", "--version", "--help", "Commands:", "lattice", "stream"]
28      )
29
```

Depois:

```
26
27  @pytest.mark.parametrize(
28      "expected", ["Options:", "--version", "--help", "Commands:", "lattice", "stream"]
29  )
30  def test_help_output_lists(expected):
31      result = CliRunner().invoke(cli, ["--help"])
32      assert expected in result.output
33
```

Por que era um test smell: A asserção usava *all(v in result.output for v in [...])*. O laço dentro do teste junta seis verificações em uma só e, quando falha, não diz qual opção faltou na ajuda.

Estratégia do CATSPY aplicada: Extrair cada caso do laço para um teste próprio. No *pytest* isso é feito com @pytest.mark.parametrize: o novo *test_help_output_lists* roda uma vez por opção esperada, e *test_help_output* ficou só com as verificações do nome do programa e do cabeçalho.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	4	3 (test_help_output) e 2 (test_help_output_lists)
SLOC do arquivo de teste (raw_por_arquivo_e_total)	297	299

R10 – ***Verbose Test (tests/test_cli.py::test_cli_output_format)***

Commit 7da8ff0 · Levi Gomes Ferreira

Antes:

```python

def test_cli_output_format(testdir):
    with TemporaryDirectory() as tempdir:
        infile = os.path.join(testdir, "health.pdf")

        runner = CliRunner()

        # json
        outfile = os.path.join(tempdir, "health.json")
        result = runner.invoke(
            cli,
            ["stream", "--format", "json", "--output", outfile, infile],
        )
        assert result.exit_code == 0, f"Output: {result.output}"

        # excel
        outfile = os.path.join(tempdir, "health.xlsx")
        result = runner.invoke(
            cli,
            ["stream", "--format", "excel", "--output", outfile, infile],
        )
        assert result.exit_code == 0, f"Output: {result.output}"

        # html
        outfile = os.path.join(tempdir, "health.html")
        result = runner.invoke(
            cli,
            ["stream", "--format", "html", "--output", outfile, infile],
        )
        assert result.exit_code == 0, f"Output: {result.output}"

        # markdown
        outfile = os.path.join(tempdir, "health.md")
        result = runner.invoke(
            cli,
            ["stream", "--format", "markdown", "--output", outfile, infile],
        )
        assert result.exit_code == 0, f"Output: {result.output}"

        # zip
        outfile = os.path.join(tempdir, "health.csv")
        result = runner.invoke(
            cli,
            [
                "stream",
                "--zip",
                "--format",
                "csv",
                "--output",
                outfile,
                infile,
            ],
        )
        assert result.exit_code == 0, f"Output: {result.output}"

```

Depois:

```python

    @pytest.mark.parametrize(
        "fmt,ext",
        [("json", ".json"), ("excel", ".xlsx"), ("html", ".html"), ("markdown", ".md")],
    )
    def test_cli_output_format(testdir, tmp_path, fmt, ext):
        infile = os.path.join(testdir, "health.pdf")
        outfile = str(tmp_path / f"health{ext}")
        runner = CliRunner()
        result = runner.invoke(
            cli,
            ["stream", "--format", fmt, "--output", outfile, infile],
        )
        assert result.exit_code == 0, f"Output: {result.output}"


    def test_cli_output_format_zip(testdir, tmp_path):
        infile = os.path.join(testdir, "health.pdf")
        outfile = str(tmp_path / "health.csv")
        runner = CliRunner()
        result = runner.invoke(
            cli,
            [
                "stream",
                "--zip",
                "--format",
                "csv",
                "--output",
                outfile,
                infile,
            ],
        )
        assert result.exit_code == 0, f"Output: {result.output}"

```

Por que era um test smell: O teste tinha cerca de 50 linhas, com quatro blocos quase idênticos (*json, excel, html, markdown*) e um caso diferente (--zip), todos dentro do mesmo TemporaryDirectory. Um teste longo e repetitivo é difícil de ler, e uma falha no meio impede que os casos seguintes rodem.

Estratégia do CATSPY aplicada: Dividir em testes menores e focados, como sugere o catálogo. A adaptação para o *pytest* foi usar @pytest.mark.parametrize para os quatro formatos e criar *test_cli_output_format_zip* para o --zip; o *TemporaryDirectory* deu lugar à *fixture tmp_path*.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	6	2 (test_cli_output_format) e 2 (test_cli_output_format_zip)
SLOC do arquivo de teste (raw_por_arquivo_e_total)	314	302

R28 – *Unknown Test (tests/test_per_page.py::test_per_page_invalid_kwarg_rejected)*

Commit aabc69e · Pablo Vinicios da Silva Araújo

Antes:

```
44
45 ••• def test_per_page_invalid_kwarg_rejected(foo_pdf):
46         """Unknown parser kwarg in per_page raises ValueError (same path as global)."""
47         with pytest.raises(ValueError, match="cannot be used with flavor"):
48             camelot.read_pdf(
49                 foo_pdf,
50                 flavor="lattice",
51                 per_page={1: {"row_tol": 5}},  # row_tol is stream/network-only
52             )
53
```

Depois:

```
44
45 ••• def test_per_page_invalid_kwarg_rejected(foo_pdf):
46         """Unknown parser kwarg in per_page raises ValueError (same path as global)."""
47         with pytest.raises(ValueError) as excinfo:
48             camelot.read_pdf(
49                 foo_pdf,
50                 flavor="lattice",
51                 per_page={1: {"row_tol": 5}},
52             )
53
54         assert "cannot be used with flavor" in str(excinfo.value)
55
```

Por que era um test smell: O teste não tinha nenhum assert; a verificação estava só no *match=* do *pytest.raises*. Na prática a mensagem já era conferida, mas a verificação não aparece como asserção, e o TEMPY só reconhece linhas com *assert*.

Estratégia do CATSPY aplicada: Introduzir uma asserção explícita do resultado esperado. A adaptação foi capturar a exceção com *pytest.raises(ValueError) as excinfo* e verificar a mensagem com *assert "cannot be used with flavor" in str(excinfo.value).*

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	1	2
SLOC do arquivo de teste (raw_por_arquivo_e_total)	45	46

R38 – *Programming Paradigms Blend (tests/test_image_conversion_backend.py* (nível do módulo))

Commit 62adc07 · Pablo Vinicios da Silva Araújo

Antes:

```
18
19  class PopplerBackendError:
20      def convert(self, pdf_path, png_path, page=1):
21          raise ValueError("Image conversion failed")
22
23
24  class GhostscriptBackendError:
25      def convert(self, pdf_path, png_path, page=1):
26          raise ValueError("Image conversion failed")
27
28
29  class GhostscriptBackendNoError:
30      def convert(self, pdf_path, png_path, page=1):
31          pass
32
33
34  class PdfiumBackendError:
35      def convert(self, pdf_path, png_path, page=1):
36          raise ValueError("Image conversion failed")
37
```

Depois:

tests/test_image_conversion_backend.py

```
3   from camelot.backends import ImageConversionBackend
4   from tests.doubles import GhostscriptBackendError
5   from tests.doubles import GhostscriptBackendNoError
6   from tests.doubles import PdfiumBackendError
7   from tests.doubles import PopplerBackendError
8
```

tests/doubles.py (arquivo novo)

```python
"""Test doubles shared by the test modules.

Kept apart from the test files so each test module sticks to one style
(plain test functions) instead of mixing module-level classes in.
"""


class PopplerBackendError:
    def convert(self, pdf_path, png_path, page=1):
        raise ValueError("Image conversion failed")


class GhostscriptBackendError:
    def convert(self, pdf_path, png_path, page=1):
        raise ValueError("Image conversion failed")


class GhostscriptBackendNoError:
    def convert(self, pdf_path, png_path, page=1):
        pass


class PdfiumBackendError:
    def convert(self, pdf_path, png_path, page=1):
        raise ValueError("Image conversion failed")
```

Por que era um test smell: O arquivo misturava quatro classes (backends falsos usados como dublês) com uma fixture e funções de teste. O CATSPY considera smell essa mistura de estilos procedural e orientado a objetos no mesmo arquivo de teste, porque dificulta entender como o teste está organizado.

Estratégia do CATSPY aplicada: Não misturar paradigmas no mesmo arquivo de teste. Como as classes não eram classes de teste, e sim dublês, não fazia sentido transformar os testes em classes. Mantivemos o estilo do projeto (funções de teste) e movemos os dublês para o novo módulo *tests/doubles.py*, que o arquivo de teste passou a importar.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	não se aplica (smell de nível de módulo)	não se aplica (smell de nível de módulo)

Métrica	Antes	Depois
SLOC do arquivo de teste (raw_por_arquivo_e_total)	49	41

R40 – *Non-Functional Statement (tests/test_errors.py::test_lattice_no_convert_method)*

Commit 41bfcdb · Pablo Vinicios da Silva Araújo

Antes:

```
141
142 ··· def test_lattice_no_convert_method(foo_pdf):
143         class ConversionBackend:
144             pass
145
146         message = "must implement a 'convert' method"
147         with pytest.raises(NotImplementedError, match=message):
148             camelot.read_pdf(
149                 foo_pdf, flavor="lattice", backend=ConversionBackend(), use_fallback=False
150             )
151
```

Depois:

```
141
142 ··· def test_lattice_no_convert_method(foo_pdf):
143         class BackendWithoutConvert:
144             """Backend stub propositalmente sem o método convert para testes de erro."""
145
146         message = "must implement a 'convert' method"
147         with pytest.raises(NotImplementedError, match=message):
148             camelot.read_pdf(
149                 foo_pdf, flavor="lattice", backend=BackendWithoutConvert(), use_fallback=False
150             )
151
```

Por que era um test smell: O corpo da classe *ConversionBackend* era só um *pass*, uma instrução sem efeito. A classe existe para representar um backend sem o método *convert*(), mas o pass não comunica essa intenção.

Estratégia do CATSPY aplicada: Remover a instrução sem efeito ou trocá-la por algo com finalidade. Como apagar o *pass* deixaria a classe vazia (erro de sintaxe), ele virou uma docstring que explica a intenção, e a classe foi renomeada para *BackendWithoutConvert*.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	1	1
SLOC do arquivo de teste (raw_por_arquivo_e_total)	155	154

4. Estado final e comparação

Métrica	Antes	Depois	Variação
Total de test smells (TEMPY)	75	35	−40
Testes aprovados	318	337	+19
Testes com falha	0	0	0
Cobertura total (%)	88,51	88,63	+0,12 p.p.
SLOC dos testes	8.741	8.761	+20
Complexidade ciclomática média das funções de teste	2,449	2,442	−0,007
Índice de manutenibilidade médio dos arquivos de teste	68,48	68,75	+0,27

Test smell	Antes	Depois	Variação
Unknown Test	54	34	−20
Conditional Test Logic	7	0	−7
Exception Handling	7	0	−7
Verbose Test	3	1	−2
Programming Paradigms Blend	2	0	−2
Redundant Print	1	0	−1
Non-Functional Statement	1	0	−1
Total	**75**	**35**	**−40**

5. Análise dos resultados

Quais métricas melhoraram e quais pioraram?

Melhoraram: o total de *test smells* caiu de 75 para 35 (−40, ou −53%). *Conditional Test Logic, Exception Handling, Programming Paradigms Blend, Redundant Print e Non-Functional Statement* foram zerados. A complexidade ciclomática média das funções de teste caiu de 2,449 para 2,442, e o índice de manutenibilidade médio dos arquivos de teste subiu de 68,48 para 68,75. Nos testes que tinham laços ou blocos repetidos, a queda de complexidade foi grande: *test_cli_output_format foi de 6 para 2 (R10) e test_help_output de 4 para 3 (R06).*

Pioraram levemente: o SLOC dos testes subiu de 8.741 para 8.761 (+20 linhas), por causa dos decoradores @pytest.mark.parametrize, das asserções explícitas e do novo módulo *tests/doubles.py*. Nas correções de *Unknown Test*, trocar *pytest.raises(..., match=...) por pytest.raises(...) as excinfo* seguido de *assert* aumenta a complexidade da função em 1, porque o *Radon* conta cada assert como um ponto de

decisão (no R28 ela foi de 1 para 2). As médias quase não mudam porque a suíte tem mais de 350 funções nos módulos de teste e cada refatoração mexeu em poucas delas.

Alguma refatoração introduziu novos *test smells*? Quais?

Só uma, e de forma temporária. O R07 (commit 2c89c3b) dividiu *test_lattice_engine_validation*, que tinha dois laços, em dois testes. A metade dos valores inválidos manteve o nome antigo e ficou apenas com *pytest.raises(ValueError, match=...)*, sem nenhum *assert*, e o TEMPY passou a acusar *Unknown Test* nela. O commit seguinte (R08, 7afa9a5), que removeu o segundo laço da mesma função, também trocou o *match=* por um *assert* explícito, e o *smell* desapareceu.

No estado final, nenhuma refatoração deixou smell novo. Conferimos isso rodando o TEMPY antes e depois de cada um dos 40 commits, nos arquivos que cada um alterou.

O resultado dos testes ou a cobertura mudou?

Nenhum teste quebrou: 0 falhas antes e depois. O número de testes aprovados subiu de 318 para 337 porque várias refatorações de *Conditional Test Logic* e *Verbose Test* transformaram um teste com laço em um teste parametrizado, e cada caso do parametrize conta como um teste. Os 7 ignorados, os 2 xfail e o xpass continuam os mesmos.

A cobertura do pacote *camelot/* subiu de 88,51% para 88,63% (+0,12 p.p.). A pequena alta é coerente com testes que passaram a exercitar caminhos que antes não eram executados: *test_pdfium_backend_import_error* (R02) e *test_import_error* (R21) só tinham asserção dentro de um except que nunca rodava, e *test_cli_quiet* (R20) terminava com erro de uso do Click sem que ninguém verificasse.

Quais tipos de smell foram mais fáceis e mais difíceis de refatorar?

Os mais fáceis foram *Redundant Print, Non-Functional Statement* e a maioria dos *Unknown Test*, que eram mudanças locais e diretas.

Os mais difíceis foram os casos em que o *smell* escondia um teste que não testava nada: *test_pdfium_backend_import_error* (R02) e *test_import_error* (R21), cujo *assert* ficava em um *except* que nunca rodava porque o módulo já estava importado e em cache, e *test_cli_quiet* (R20), em que a opção --quiet engolia o --format e o comando terminava com erro de uso. Também deram trabalho o *Verbose Test* de *test_cli_stream* (R09), que exigiu separar vários cenários de uma função com mais de 50 linhas, e o *Conditional Test Logic* de *test_overlapping_text_preserves_adjacent_cell* (R24), que precisou de uma verificação vetorizada com pandas para manter uma mensagem de erro útil.

6. Uso do CATSPY

Como você usou o CATSPY durante a atividade?

Usamos o CATSPY em todas as ocorrências, antes de refatorar. Líamos a página do smell para confirmar que a ocorrência apontada pelo TEMPY correspondia à definição do catálogo e para escolher a estratégia de refatoração. Cada registro do diário de bordo indica a página consultada. Também usamos a severidade do catálogo para priorizar os tipos (Seção 3.1).

Quais partes do catálogo mais ajudaram (definição, efeitos negativos, exemplo com problema, exemplo refatorado)?

A definição e o exemplo refatorado. A definição ajudou a separar o que era problema real do que era só a forma como a ferramenta detecta. O exemplo refatorado deu o formato da solução: trocar o *try/except* manual pelo recurso de exceção do *framework*, no *Exception Handling*, e extrair cada caso do laço para um teste próprio, no *Conditional Test Logic*. Os efeitos negativos ajudaram a justificar as mudanças no diário e na descrição do PR.

Houve casos em que a estratégia do catálogo não se aplicava diretamente ao projeto? Como você resolveu?

Sim. Os exemplos do catálogo são em unittest (*unittest.TestCase, assertRaises, self.assert...)*, e o Camelot usa *pytest*, então quase toda estratégia precisou ser traduzida: *assertRaises* virou *pytest.raises*, e "um teste por caso" virou @pytest.mark.parametrize. Alguns casos exigiram adaptação maior:

• *Exception Handling* em *test_no_tables_found_logs_suppressed* (R03) e *test_no_tables_found_warnings_suppressed* (R04): o *try/except* não esperava exceção nenhuma, então não havia o que trocar por *pytest.raises*. A solução foi remover o bloco, porque *warnings.simplefilter*("*error*") já faz o teste falhar.

• *Unknown Test* com *pytest.raises*(..., *match*=...): o teste já verificava a mensagem, mas sem *assert*. A solução foi capturar a exceção com *as excinfo* e escrever a asserção explícita.

• *Programming Paradigms Blend*: as classes eram dublês de teste, não classes de teste. Em vez de transformar os testes em classes, movemos os dublês para *tests/doubles.py*.

• *Non-Functional Statement*: o *pass* não podia ser só apagado, porque deixaria a classe vazia. Ele virou uma *docstring*.

O que faltou ou poderia ser melhorado no catálogo?

Exemplos em *pytest*, que é o framework mais usado em projetos Python hoje, ao lado dos de unittest. Também faria falta uma orientação sobre como as ferramentas detectam cada smell: o *Unknown Test* acusa testes que verificam com *pytest.raises(match*=...), que para o pytest já é uma verificação, e saber disso antes evitaria dúvidas sobre se a refatoração era necessária. Por fim, seria útil uma seção sobre dublês de teste (stubs e fakes) no *Programming Paradigms Blend*, para diferenciar classes de apoio de classes de teste.

7. Divisão do trabalho

Integrante	Refatorações realizadas (IDs da Seção 3.2)	Quantidade
Levi Gomes Ferreira (556935)	R01 a R20 *(Redundant Print, Exception Handling, Conditional Test Logic, Verbose Test, Unknown Test)*	20
Pablo Vinicios da Silva Araújo (574229)	R21 a R40 *(Exception Handling, Conditional Test Logic, Unknown Test, Programming Paradigms Blend, Non-Functional Statement)*	20

8. Dificuldades e lições aprendidas

Quais foram as principais dificuldades (ambiente, dependências, entendimento do código, ferramentas, revisão do PR)?

Ambiente: o Ubuntu do WSL não tinha o python3-venv, então criamos o ambiente com o uv; também foi preciso instalar o *Ghostscript* e o *Poppler* para que os testes de conversão de imagem rodassem. No Windows, 24 testes são pulados pelo próprio projeto (skip_on_windows), então as métricas oficiais foram geradas no Linux (WSL) para que o antes e o depois fossem comparáveis. A cobertura mínima de 90% do projeto faz o pytest terminar com erro mesmo sem nenhuma falha, o que confunde na primeira leitura.

Ferramentas: o TEMPY só tem interface gráfica, então o usamos pelo script de extração. Ele também só reconhece asserções escritas com *assert*, o que gerou ocorrências de *Unknown Test* em testes que já verificavam a mensagem com pytest.raises(match=...).

Entendimento do código: em alguns testes foi preciso entender por que o caminho testado nunca era executado (módulos já importados e em cache, opções do Click que consumiam o argumento seguinte) antes de conseguir refatorar sem perder a intenção original.

Git e colaboração: trabalhamos com uma branch por tipo de *smell*, encadeadas e compartilhadas pela dupla. Alguns commits foram feitos na branch errada e outros deixavam testes quebrados no meio do caminho; refazê-los exigiu reescrever parte do histórico com cuidado para não afetar os commits do colega.

Revisão do PR: o CI do Camelot verifica a formatação com *ruff format*, o que só descobrimos ao preparar o PR (4 arquivos fora do padrão), e o guia de contribuição pede mensagens de commit em um formato diferente do exigido pela disciplina.

O que você aprendeu com a atividade?

Que um teste passar não significa que ele testa algo: três dos testes refatorados nunca chegaram à asserção. Que a saída de uma ferramenta de detecção precisa de julgamento humano, porque nem toda ocorrência é um problema do mesmo tamanho. Que vale rodar localmente tudo o que o CI do projeto roda antes de abrir um PR. E que commits pequenos, um por refatoração e com os testes passando em cada um, tornam o trabalho em dupla muito mais fácil de revisar e de corrigir.

9. Contribuição (*Pull Request*)

Abrimos o PR #871 (https://github.com/camelot-dev/camelot/pull/871) em 01/11/2026, a partir da branch *refactor/remove-test-smells* do *fork*, com o título "[MRG] *refactor(tests)*: *remove test smells from the test suite*". A descrição, em inglês, explica o motivo, lista as 40 refatorações por tipo de smell, destaca os testes que antes não verificavam nada e informa como tudo foi testado, além de trazer uma lista de tarefas.

Seguimos o guia de contribuição do Camelot (*docs/dev/contributing.rst*): prefixo [MRG] no título, lista de tarefas na descrição e código no padrão do projeto. Antes de abrir, rodamos o que o CI executa — *ruff check, ruff format --check e mypy camelot tests* — e a suíte completa. O *ruff format* apontou 4 arquivos fora do padrão e uma *docstring* estava em português; corrigimos com dois commits extras só na branch do PR. O PR tem 44 commits (as 40 refatorações, 2 merges entre as branches de tipo e os 2 ajustes) e altera apenas arquivos de *tests/*. Pastas de métricas e scripts de extração ficaram de fora, como pede a atividade.

Até a data deste relatório não houve resposta dos mantenedores: o PR está aberto, sem comentários, e o CI ainda não rodou, porque, para quem contribui pela primeira vez, a execução depende da aprovação de um mantenedor.

10. Observações gerais

Mensagens de commit: seguimos o formato exigido pela atividade (*refactor: remove <smell>* em <arquivo>::<teste>), que difere do recomendado pelo guia do Camelot (assunto em inglês, com até 50 caracteres). Mantivemos o formato da atividade e indicamos no PR que os commits podem ser unificados (*squash*) no *merge*.

Métricas: as do antes e as do depois foram geradas com os mesmos scripts e no mesmo ambiente (WSL Ubuntu 24.04, Python 3.12). A média de complexidade considera todas as funções com origem = teste no CSV do Radon; contando só as funções test_*, ela vai de 2,580 para 2,574. O índice de manutenibilidade passou a considerar 33 arquivos de teste em vez de 32, por causa do novo tests/doubles.py. Nos exemplos da Seção 3.3, a complexidade e o SLOC foram medidos no commit anterior e no próprio commit, para isolar o efeito de cada refatoração.

Testes: além dos números da tabela 2.2, a suíte teve 2 falhas esperadas (xfail) e 1 xpass, tanto antes quanto depois. Cada um dos 40 commits foi testado com os testes do arquivo alterado, e a suíte completa passou no fim de cada branch.

Smells restantes: ficaram 35 ocorrências (34 de Unknown Test e 1 de Verbose Test, em test_text_in_bbox_filters_and_discards_overlaps), fora do escopo das 40 escolhidas.

Pastas de métricas: o coverage cria um .gitignore dentro das pastas coverage_*_html, o que faz o git add ignorá-las; para versioná-las no fork foi preciso usar git add -f.